

ALNS on Integrated Healthcare Timetabling

Bello Renato
Politecnico di Torino
Student id: s341965
s341965@studenti.polito.it

Martin Emanuele
Politecnico di Torino
Student id: s343316
s343316@studenti.polito.it

Terenziani Andrea
Politecnico di Torino
Student id: s284817
s284817@studenti.polito.it

Abstract—This project applies a neighborhood search heuristic known as *Adaptive Large Neighborhood Search (ALNS)* to a hospital organization problem¹. The aim of the problem is to schedule and allocate hospital resources—such as *rooms, patients, surgeons, operating theatres, and nurses*—while strictly respecting a set of hard constraints and minimizing the cost associated with soft constraint violations. We first generate an initial feasible solution, and then use the ALNS framework to explore its neighborhood in search of improved solutions.

I. PREPROCESSING AND DATA STRUCTURES

To evaluate our work, we used a set of 10 JSON files as test instances. All the results reported in this report refer to these test files. The core of the program can be found in the file `main.py`, which is included in the repository associated with this report. In this file, the program loads a selected test instance and invokes the `preprocess` function (defined in `utils.py`). This function processes the input data and generates all the necessary dictionaries to initialize the internal data structures.

The folder `instances` contains all the main classes used to model the problem:

- **Occupant:** Represents individuals who are already hospitalized from day 0. These are not surgical patients, but require supervision by nurses during their stay.
- **Patient:** Similar to *Occupant*, but with additional attributes:
 - *compatible rooms*: the rooms where the patient can be assigned;
 - *surgeon*: the surgeon assigned to perform the surgery;
 - *mandatory*: indicates whether the patient requires urgent care;
 - *surgery release day*: the earliest day the patient can be admitted;
 - *surgery due day*: the latest admission day for urgent patients.
- **Nurse:** Stores information about each nurse in the hospital.
- **Surgeon:** Stores information about each surgeon in the hospital.
- **Rooms:** Defined in `hospital.py`, this class manages room-related data. It includes the following methods:

- `add_occupant`: adds an occupant to the hospital, typically used during the initial solution construction;
- `add_patient`: similar to `add_occupant`, but for patients;
- `remove_patient`: removes a patient from the hospital and is used during the *destroy* phase of the ALNS algorithm.

- **Theaters:** Defined in `hospital.py`, this class manages data regarding the operating theaters.
- **Time:** Also in `hospital.py`, this class stores scheduling information, including the total number of days and shifts to be considered.
- **Solution:** Represents a candidate solution to the problem. It consists of two main components:
 - *nurse schedule*: a dictionary specifying which room each nurse is assigned to for each shift and day;
 - *patient schedule*: composed of two dictionaries that track the patient’s admission day, assigned operating theater, and room allocation.
- **Problem:** This class takes a `Solution` object and verifies whether it satisfies all hard constraints. It also evaluates the soft constraints, interpreting them as part of a cost function to be minimized.

II. HARD AND SOFT CONSTRAINTS

The **Problem** class includes two main methods. The first checks whether a solution is feasible with respect to the hard constraints, which must always be satisfied. These are:

- **H1:** No gender mixing is allowed within a room; all patients in a room must have the same gender.
- **H2:** Each patient must be assigned to one of their compatible rooms and must remain in the same room for the entire duration of their stay.
- **H3:** Each surgeon must not exceed their maximum daily working time for performing surgeries.
- **H4:** Each operating theater must not exceed its maximum daily operational time.
- **H5:** All mandatory patients must be admitted to the hospital.
- **H6:** Each patient must be admitted within their allowed admission window. For mandatory patients, this means between the release day and the due day. Non-mandatory patients, if admitted, must be admitted on or after their release day.

¹<https://ihtc2024.github.io/>

- **H7:** Room occupancy must not exceed the maximum capacity of the room.

All these constraints are defined in the original problem specification². In addition to those, we introduced a custom constraint to better reflect the hospital management setting:

- **H8:** Every non-empty room must be supervised by exactly one nurse.

The second method in the **Problem** class evaluates the soft constraints, which are not mandatory but contribute to the solution's cost if violated. These are:

- **S1:** Minimize the maximum age difference among patients within the same room.
- **S2:** Minimize the number of nurses assigned to rooms where their skill level is insufficient (the skill level of a room is computed as the maximum skill level required by the patients/occupants in that room).
- **S3:** Minimize the number of nurses that providers care to each given patient or occupant.
- **S4:** Minimize the discrepancy between the required workload in each room and the capacity of the assigned nurse for every day and shift.
- **S5:** Minimize the number of operating theaters that are open each day.
- **S6:** Minimize the number of different operating theaters assigned to the same surgeon for each day.
- **S7:** Minimize the length of hospital stays, i.e., the number of days between a patient's admission and discharge.
- **S8:** Minimize the number of non-mandatory patients who are not admitted to the hospital.

III. ADAPTIVE LARGE NEIGHBOURS SEARCH IMPLEMENTATION

Into the folder `solver` are stored all the files that are related to the solution generation and the optimization.

Algorithm 1 Adaptive Large Neighborhood Search (ALNS)

```

1: Initialize solution  $s \leftarrow s_0$ 
2: Set best solution  $s^* \leftarrow s$ 
3: Initialize weights for destroy and repair operators
4: for each iteration do
5:   Select a destroy operator  $d$  based on weights
6:   Select a repair operator  $r$  based on weights
7:   Generate new solution  $s' \leftarrow r(d(s))$ 
8:   if  $\text{accept}(s, s')$  then
9:      $s \leftarrow s'$ 
10:    if  $f(s') < f(s^*)$  then
11:       $s^* \leftarrow s'$ 
12:    end if
13:  end if
14:  Update operator weights
15: end for
16: return  $s^*$ 

```

Starting from the first line of the pseudocode, we create a file called `initial_solution.py`. Into this file is contained a function called `initial_solution` that generate the starting solution. The main problem of this function is to generate the schedule of the patient, to aim that we decided to give priority to the mandatory patient, those patient are sorted by the difference between their *due surgery day* and *release due day*, after that we started to schedule the patients following the sorting that we have done before. With this strategy we are able to schedule all the mandatory patients, after that the function tries to schedule also the not mandatory patients that remains. For the schedule of the nurses we simply cycle over the days and shifts and assigned a nurse to each room (taking care of all the possible turns of the nurses).

For implementing ALNS we constructed the *Repair* and the *Destroy* operators, those are contained into the files `Repair.py` and `Destroy.py`.

Here presented all the *destroy* operations:

- **A:** Select randomly a day, shift and room and remove all the other nurses from the schedule in the solution that don't work during that day and shift in that room selected.
- **B:** Select randomly a day, shift and room and remove all the other nurses from the schedule in the solution that work during that day and shift in that room selected.
- **C:** Select randomly some days, shifts and rooms and remove all the nurses from the schedule in the solution that work during all the combinations of the days, shifts and rooms selected.
- **D:** Select randomly a patient from the solution schedule and remove all the other patient from schedule.
- **E:** Select randomly a patient from the solution schedule and remove that patient.
- **F:** Select randomly a group of patients and remove them.
- **G:** Is a mix of destroy operations *A* and *D*.
- **H:** Is a mix of destroy operations *A* and *E*.
- **I:** Is a mix of destroy operations *A* and *F*.
- **L:** Is a mix of destroy operations *B* and *D*.
- **M:** Is a mix of destroy operations *B* and *E*.
- **N:** Is a mix of destroy operations *B* and *F*.
- **O:** Is a mix of destroy operations *C* and *D*.
- **P:** Is a mix of destroy operations *C* and *E*.
- **Q:** Is a mix of destroy operations *C* and *F*.

All the *repair* operators are divided into two part: repairing the schedule of the patients, repairing the schedule of the nurses. The first part is common to all the *repair* operators and it's based on schedule of all the mandatory patients that where destroyed, for aim that the program removes not mandatory patients from the hospital until it finds a place for all the destroyed mandatory patients, after that it tries to find place for the not mandatory patients that still not in the hospital. The second part of the *repair* operations we found 4 different methods for repairing the schedule of the nurses:

- **A:** With the same logic used for finding the initial solution, we cycle over the days and shifts and assigned a nurse to each room that is nurses empty (because

²<https://ihtc2024.github.io/>

the destroy operator might have destroyed some nurses schedule).

- **B:** The logic is the same of the case *A*, but we try to assign a nurse to a room that minimize the difference between the skill level required in the room and the skill level of the nurse.
- **C:** The logic is the same of the case *A*, but we assign a nurse to a room trying to minimize the exceeded workload of the nurse.
- **D:** It's a mix of the logic *B* and *C*.

We defined the class **ALNS** into the file `ALNS.py`, into that we implemented the method `solve`, that given an initial solution compute the optimization using the algorithm presented above, as stopping criteria we selected the *reaching of the maximum number of iterations selected*. We initialize the weights uniformly and computed the probabilities of the operators using the following formulas:

$$p(d_i) = \frac{w(d_i)}{\sum_i w(d_i)}, \quad p(r_i) = \frac{w(r_i)}{\sum_i w(r_i)} \quad (1)$$

The function $accept(s, s')$ used in the line 8 of the algorithm return TRUE if:

- $f(s) < f(s')$
- A *Bernoulli* random variable with parameter $p = e^{-\gamma t}$ returns 1, where t is the current number of iteration and $\gamma > 0$ is a tuning parameter.

For adjusting the weights are defined the following quantities:

- $u(h)$ is the number of times that the operator h has been used during the optimization.
- $s(h)$ is a quantity called *Success* of the operator h , it is initialize at 0 for each operator.

After using the operator h , we increase the Success $s(h)$ by the quantities:

- δ_1 if the solution found is the best one so far.
- δ_2 if the new solution improves the current solution.
- δ_3 if the new solution is accepted but doesn't improve the current solution.
- δ_4 if the new solution in rejected.

Those quantities are tuned, for good results should be $\delta_1 \geq \delta_2 \geq \delta_3 \geq \delta_4$.

At line 14 we update the weight using the following formula:

$$w(h) = \begin{cases} (1 - \rho)w(h) + \rho \frac{s(h)}{u(h)} & u(h) > 0 \\ (1 - \rho)w(h) & u(h) = 0 \end{cases} \quad (2)$$

Where $\rho \in [0, 1]$ is a tuning parameter.

IV. RESULTS AND COMMENTS

For all our tests we setted the seed of the random generator as 284817. We tested the program over the 10 JSON files available, we tried to tune the parameters of the solver obtaining the following results:

Test	Iterations	Op. Value	ALNS Value
1	150000	3177	4002
2	50000	1583	2375
3	10000	10184	11082
4	250000	2332	3103
5	1000	15713	16922
6	40000	18558	24905
7	30000	17048	20243
8	30000	24947	26851
9	35000	20492	25600
10	10000	49444	84594

TABLE I: Table of the results obtained

Test	ρ	$[\delta_1, \delta_2, \delta_3, \delta_4]$	γ
1	0.001	[8, 5, 1.5, 0.5]	0.9
2	0.9	[8.5, 8, 4, 0.5]	0.3
3	0.1	[8, 5, 1.5, 0.5]	1.5
4	0.001	[8, 5, 1.5, 0.5]	0.9
5	0.3	[8, 3, 2, 0.5]	0.5
6	0.001	[8, 5, 1.5, 0.5]	0.9
7	0.001	[8, 5, 1.5, 0.5]	0.9
8	0.001	[8, 5, 1.5, 0.5]	0.9
9	0.001	[8, 5, 1.5, 0.5]	1.5
10	0.001	[8, 5, 1.5, 0]	0.9

TABLE II: Table of the tuning parameters

Test	Total time	Time for destroy	Time for repair
1	7705.14	3952.54	3752.61
2	705.77	358.60	347.17
3	507.44	260.36	247.07
4	22526.26	11667.99	10858.27
5	23.05	11.59	11.46
6	2625.35	1307.61	1317.74
7	4262.67	2200.35	2062.32
8	5175.67	2665.95	2509.72
9	9513.41	4971.15	4542.25
10	7937.11	3895.94	4041.18

TABLE III: Table of the times of optimization

The following pages report the plots of the *ALNS* algorithm:

- In the first page we find, for each test, the plot of the iterations of the *ALNS* solver vs the function value of the current solution found by the solver.
- In the second page we find, for each test, the plot of the iterations of the *ALNS* solver vs the probabilities of each *destroy operator*.
- In the last page we find, for each test, the plot of the iterations of the *ALNS* solver vs the probabilities of each *repair operator*.

We can observe, from the following plots, that *destroy operator Q* is the most commonly used by the *ALNS* algorithm, that is because is the most complete one. From the plots of the *repair operators*, we can see that all of them, for most of the cases, are equally used with the same probabilities. A possible improve that can be apply to the program is to add a restart point if the solution doesn't improve any more during the optimization (following a logic similar to the *GRASP solver*), with this logic we have more solutions and more neighbours to explore. Another improve would be generating different starting solutions, running in the *ALNS* algorithm in parallel and select the best solution.





